

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re C D E

Bases de données relationnelles : initiation

Module 1 : Environnement numérique, sécurité, multimédia et gestion des données

Cette fiche propose des exercices de synthèse mobilisant la modélisation, le langage SQL (DDL/DML) et l'analyse critique d'architectures SGBD.

EXERCICE 1 Modélisation et contraintes d'intégrité complexes

On souhaite informatiser la gestion des stages pour un lycée technique. Les règles de gestion sont :

- Un Stagiaire est identifié par un Matricule, a un Nom, Prénom, DateNaissance, Email (unique), Spécialité (ex : 'Réseaux', 'Développement').
 - Une Entreprise a un SIREN (clé), RaisonSociale, Adresse, SecteurActivité.
 - Un TuteurEntreprise (employé de l'entreprise) a un IdTuteur, Nom, Prénom, Téléphone, Email. Il appartient à une seule entreprise.
 - Un Professeur (tuteur pédagogique) a un IdProf, Nom, Prénom, Matière.
 - Un Stage lie un stagiaire, une entreprise, un tuteur entreprise et un professeur sur une Période (DateDébut, DateFin). Une Note (sur 20, par défaut NULL) et une Appréciation sont renseignées à la fin.
 - Contraintes métier : Un stagiaire ne peut faire qu'un stage à la fois (pas de chevauchement de périodes). Un tuteur entreprise ne peut encadrer plus de 3 stagiaires simultanément. La note doit être entre 0 et 20.
1. Proposer un MCD (Modèle Conceptuel de Données) respectant la notation Merise (entités, associations, cardinalités, identifiants relatifs). Justifier le choix des identifiants pour l'entité faible Stage.
 2. Traduire ce MCD en MLD (Modèle Logique de Données) relationnel. Souligner les clés primaires et indiquer les clés étrangères par une flèche →.
 3. Écrire les instructions SQL (DDL) de création des tables `Entreprise`, `TuteurEntreprise` et `Stage` en intégrant toutes les contraintes d'intégrité (PK, FK, UNIQUE, NOT NULL, CHECK, DEFAULT) au niveau colonne ou table. Utiliser des types standards (INTEGER, VARCHAR, DATE).
 4. La direction décide d'archiver les stages anciens. Écrire l'instruction `ALTER TABLE` pour ajouter une colonne `Archive` `BOOLEAN` `DEFAULT FALSE` à la table `Stage`, puis l'instruction pour créer un index sur (`Archive`, `DateDebut`).

EXERCICE 2 Manipulation de données et cohérence transactionnelle

On considère la base `GestionStages` issue de l'exercice précédent, peuplée de données. On souhaite automatiser la cloture d'un stage.

1. Écrire une transaction SQL (bloc `BEGIN ... COMMIT`) qui, pour un `NumStage` donné :
 - (a) Vérifie que le stage existe et n'est pas déjà clôturé (`Note IS NOT NULL`).
 - (b) Met à jour la `Note` et l'`Appréciation`.
 - (c) Décrémente de 1 le compteur `NbStagiairesEnCours` de la table `TuteurEntreprise` (colonne à supposer existante).
 - (d) Signale une erreur (`ROLLBACK`) si la note n'est pas dans `[0,20]` ou si le tuteur n'existe plus.
2. Expliquer pourquoi l'isolation des transactions (niveau `READ COMMITTED` vs `SERIALIZABLE`) est cruciale lors de l'affectation simultanée de deux stagiaires au même tuteur (contrainte "max 3"). Illustrer par un scénario d'anomalie possible (perte de mise à jour ou lecture sale).
3. Proposer une requête `SELECT` utilisant des jointures internes (`INNER JOIN`) et une jointure externe (`LEFT JOIN`) pour lister : `Matricule`, `NomStagiaire`, `RaisonSociale`, `NomTuteurEnt`, `NomProf`, `Note` pour tous les stagiaires, même ceux sans stage affecté. Trier par spécialité puis nom.
4. Écrire une requête d'agrégation donnant, par entreprise et par année de début de stage, le nombre de stagiaires, la note moyenne et la note maximale. Ne conserver que les groupes ayant au moins 2 stagiaires.

EXERCICE 3 Architecture SGBD, sécurité et critique

1. Décrire les cinq composants principaux de l'architecture interne d'un SGBD relationnel (ex : Moteur de stockage, Gestionnaire de buffer, Optimiseur de requêtes, Gestionnaire de transactions, Catalogue/Système). Pour chacun, préciser son rôle précis vis-à-vis des fonctions de définition (DDL) et de manipulation (DML).
2. Comparer les approches centralisée (monoposte/serveur unique) et distribuée (fragmentation/réplication) pour l'hébergement de la base `GestionStages` à l'échelle académique (plusieurs lycées). Citer 2 avantages et 2 inconvénients majeurs pour chaque architecture concernant : la disponibilité, la cohérence des données (contraintes d'intégrité référentielles), et la latence d'accès.
3. Critiquer la proposition suivante : "Pour simplifier, on supprime les clés étrangères au niveau physique (SGBD) et on gère l'intégrité référentielle uniquement dans le code de l'application cliente (PHP/Python)." Argumenter sur les risques d'anomalies (insertion orpheline, suppression en cascade non gérée), la performance (vérifications

applicatives vs optimiseur), et la maintenance (évolution du schéma).

4. Citer trois SGBD relationnels majeurs du marché (un propriétaire, un open-source standard, un embarqué/léger). Pour chacun, donner un cas d'usage typique justifié (ex : haute concurrence, embarqué, analytique).

EXERCICE 4 Normalisation et anomalies de mise à jour

On récupère un fichier “plat” (table non normalisée) `Inscriptions` utilisé par l'administration pour suivre les inscriptions aux modules de formation continue :

```
Inscriptions(IdInscri, DateInscri, IdStagiaire, NomStag, EmailStag,  
            IdModule, LibModule, NbHeures, IdSalle, CapaciteSalle,  
            IdFormateur, NomForm, SpecialiteForm, TarifModule)
```

Chaque inscription concerne un stagiaire, un module, une salle et un formateur. Un module a un tarif unique. Un formateur a une spécialité unique. Une salle a une capacité fixe.

1. Identifier la clé primaire de cette relation. Lister toutes les dépendances fonctionnelles (DF) non triviales évidentes.
2. Démontrer que cette relation n'est pas en 2NF, puis pas en 3NF. Citer explicitement les DF responsables (dépendance partielle, dépendance transitive).
3. Décrire précisément les trois types d'anomalies (insertion, suppression, modification) qui se produisent sur cette table. Illustrer chaque anomalie par un exemple concret de tuple.
4. Proposer un schéma relationnel normalisé en 3NF (ou BCNF si applicable). Donner les tables avec clés primaires/étrangères. Vérifier l'absence de dépendances transitives résiduelles.
5. Écrire les requêtes `CREATE TABLE` pour deux de vos nouvelles tables, en définissant des contraintes `CHECK` sur `CapaciteSalle > 0` et `TarifModule >= 0`, et une contrainte `UNIQUE` sur `EmailStag`.